



El Orquestador flujo_repo_a_skill.py: Utilidad y Diseño

➤ Capítulo 3 de la Fase 1b | Repositorio → Skill de Referencia de Código/API

Ficha Técnica

📁 Capa:	Orquestación (Layer 2)	📄 Directiva:	directives/repo_a_skill.yaml
➤ Invocación:	python3 flujo_repo_a_skill.py -repo <url dir>	💰 Créditos:	Paso 3 (síntesis) openrouter_chat
📄 Perfil de salida:	referencia_codigo	✅ Modelo default:	deepseek/deepseek-v4-pro

Índice

1. 🎯 ¿Qué es el Orquestador y Para Qué Sirve?	1
1.1. Por qué es necesario (el problema que resuelve)	1
2. 🏗️ La Arquitectura de 3 Capas de este Flujo	2
3. 📊 Entradas, Salidas y Flags Principales	2
3.1. Entrada requerida	2
3.2. Salidas que produce	2
3.3. Flags más útiles	3
4. 📋 Los 8 Pasos del Flujo	3
5. 📄 Trazabilidad, Errores y Recuperación	3
5.1. Estado persistente: .tmp/run_state.json	4
5.2. Códigos de salida y política de reintentos	4
5.3. Casos límite gestionados (extracto de la directiva)	4
6. 🧪 Ejemplo Real: pallets/itsdangerous	5
7. ➤ Guía de Invocación Rápida	6

1 🎯 ¿Qué es el Orquestador y Para Qué Sirve?

flujo_repo_a_skill.py es la **capa de orquestación** de la Fase 1b: el *middleware* que conecta **la intención del usuario** (“quiero un skill a partir de este repositorio”) con **los scripts deterministas de ejecución** de execution/. Convierte un **repositorio de código** (URL de GitHub o directorio local) en un **skill global de referencia de código/API**, instalado en ~/.config/opencode/skills/<nombre>/.

Concepto clave

En una frase: el orquestador orquesta 5 scripts de ejecución en una secuencia de 8 pasos, mantiene el **estado** en .tmp/run_state.json, decide el **tier LLM de forma determinista** y aplica las **políticas de recuperación** ante fallos. No “razona” la lógica de negocio: simplemente la *dirige*.

1.1 Por qué es necesario (el problema que resuelve)

- **Un LLM alucina las APIs** de una librería si solo “recuerda” haberlas visto. Un skill formal y verificado ancla la referencia a la fuente real.
- **La fiabilidad compuesta se degrada:** si 5 pasos tienen 90 % de acierto cada uno, el resultado global cae a ≈59 %. El orquestador empuja la complejidad a scripts deterministas (90 %+) y mantiene la toma de decisiones “delgada”.
- **Reproducibilidad:** misma entrada → misma salida. El flujo se puede re-ejecutar sin depender de la memoria del chat; el estado en disco lo permite retomar.
- **Separación de responsabilidades** (arquitectura de 3 capas): la directiva documenta *qué* hacer, el orquestador decide *cuándo* y *cuál* script, y los scripts saben *cómo* ejecutar.

2 La Arquitectura de 3 Capas de este Flujo

Capa	Artefacto	Rol
Directiva	directives/repo_a_skill.yaml	SOP determinista: goal, 8 pasos, 7 edge cases, política de reintentos y metadata de coste.
Orquestación	flujo_repo_a_skill.py	Entrevista, orquesta scripts, decide el modelo (enrutador), valida salidas intermedias, gestiona errores y notifica.
Ejecución	execution/extraer_repo_github.py execution/sintetizar_skill.py execution/validar_skill_formulas.py execution/generar_latex_skill.py execution/instalar_skill.py	extraer_repo_github.py: clona/filtra/concatena. sintetizar_skill.py: destila el corpus con el LLM. validar_skill_formulas.py: valida bloques Python sin errores. generar_latex_skill.py: reporte PDF. instalar_skill.py: copia a global.

Tabla 1: Responsabilidades de las 3 capas en la Fase 1b.

Regla de oro del orquestador

El orquestador no raspa, no destila, no valida y no instala por sí mismo. Cada paso delegado llama a un script de execution/ con CLI estricta; las salidas se validan (JSON, códigos de salida) antes de avanzar. Esto garantiza que “el LLM del chat” no introduzca lógica arbitraria en el flujo.

3 Entradas, Salidas y Flags Principales

3.1 Entrada requerida

- `-repo` (obligatorio): URL de GitHub (<https://github.com/owner/repo>) o ruta a un directorio local. Es la **única entrada indispensable**.
- Opcionales: `-tema`, `-nombre`, `-idioma`. Si se omiten, el orquestador hace una **entrevista ligera interactiva** y persiste la respuesta en `.tmp/entrevista_skill_<nombre>.json`.

3.2 Salidas que produce

Salida	Dónde
Skill global instalado	<code>~/.config/opencode/skills/<nombre>/</code> (SKILL.md + references/)
Copia intermedia de revisión	<code>.tmp/skill_<nombre>/</code>
Registro de la entrevista	<code>.tmp/entrevista_skill_<nombre>.json</code>
Texto extraído del repo	<code>.tmp/repo_<nombre>_texto/texto_completo.txt + indice.json</code>
Reporte LaTeX del skill	<code>docs/SKILL/<nombre>/<nombre>.tex + .pdf</code>
Estado de la ejecución	<code>.tmp/run_state.json</code>

Tabla 2: Artefactos de entrada/salida del flujo.

3.3 Flags más útiles

4 Los 8 Pasos del Flujo

Flag	¿Créditos?	Qué hace
-dry-run	Según paso	Genera y valida el skill sin instalarlo en global.
-modelo <id>	Sí (openrouter)	Override explícito del tier decidido por el enrutador.
-incluir/-excluir	No	Globs para acotar qué archivos del repo se destilan.
-reflexion N	Sí (re-síntesis)	Re-sintetiza hasta $N \leq 3$ veces si hay bloques inválidos.
-validar-estricto	No	Aborta la instalación si la validación halla errores.
-estructura-max-tokens N	No	Presupuesto de salida del ensamblaje (auto: 32768 en deepseek, 8192 resto).
-no-latex	No	Omite el reporte LaTeX del skill.
-no-alert	No	Omite la alerta audible final.

Tabla 3: Catálogo de flags del orquestador y efectos.

#	Qué hace	Script	Créditos / Notas
1	Entrevista ligera + validación de fuente	orquestador	0 créditos; persiste entrevista.
2	Extracción: clona/filtra/concatena	extraer_repo_github.py	0; determinista; genera texto_completo.txt + indice.json.
3	Decisión determinista de tier	enrutador.py -task contexto_masivo	0; mide tokens y elige modelo en código.
4	Destilación LLM del skill	sintetizar_skill.py -perfil referencia_codigo	Sí ; genera SKILL.md + 6 references de API.
5	Validación neuro-simbólica	validar_skill_formulas.py -perfil referencia_codigo	0; sólo riesgo de Sistema bloquea; reflexión opcional.
6	Reporte LaTeX infográfico	generar_latex_skill.py	0; PDF en docs/SKILL/<nombre>/.
7	Revisión humana	orquestador + usuario	0; aprueba o re-sintetiza (≤ 3).
8	Instalación global + aviso	instalar_skill.py + alert_user.py	0; recuerda reiniciar opencode.

Tabla 4: Secuencia de 8 pasos del orquestador flujo_repo_a_skill.py.

Fortaleza de diseño

¿Por qué solo el Paso 4 consume créditos? Siguiendo el principio “*orquestación gratis, ejecución medida*”, la extracción, el enrutamiento (decisión 100 % local), la validación, el reporte y la instalación son deterministas y no llaman a ninguna API de pago. La única fase probabilística necesaria (la destilación por LLM) queda acotada a un único script con -modelo y -estructura-max-tokens auditable.

5 Trazabilidad, Errores y Recuperación

5.1 Estado persistente: .tmp/run_state.json

Tras cada paso exitoso el orquestador guarda: run_id, directive, fuente, nombre, perfil, tier/modelo, dry_run, current_step, steps_completed, steps_failed y marca temporal. Esto permite saber **en qué paso quedó** un run y diagnosticar fallos sin re-ejecutar a ciegas.

5.2 Códigos de salida y política de reintentos

- 0 → flujo completado (con -dry-run también es 0).
- 1 → fallo fatal en extracción, síntesis o instalación (se aborta y alerta).
- 3 → bloques inválidos (vía validador), relevante con -validar-estricto.

⚠ Punto crítico de diseño

Retry budget: máximo **3 reintentos** por tarea (lo impone el validador y el bucle -reflexion). Si se agotan, el flujo **escala al usuario** en lugar de seguir intentando a ciegas. Un fallo de API o de JSON se diagnostica, se corrige la causa y se reintenta; nunca se "fuerza" la salida.

5.3 Casos límite gestionados (extracto de la directiva)

Caso	Recuperación
Repo privado o 404	git clone falla; ofrecer ruta local o GITHUB_TOKEN.
Repo sin código/docs tras filtrar	Código 2 del extractor; sugerir -incluir.
Repo enorme / coste alto	Antes de la síntesis se muestran tokens medidos; proponer acotar con globs.
Bloques Python inválidos	Avisar; -validar-estricto aborta; -reflexion re-sintetiza (créditos).
El repo cambia o la ruta desaparece	Cada corrida re-clona; si la ruta local no existe, código 1 y fin.

Tabla 5: Casos límite y su protocolo de recuperación.

6 Ejemplo Real: pallets/itsdangerous

La primera prueba E2E del orquestador se ejecutó sobre el repositorio público <https://github.com/pallets/itsdangerous> (librería de firma de datos del proyecto Pallets, 32 archivos de código y documentación):

Resultados de la ejecución real

Volumen de entrada:	32 archivos incluidos / 8 excluidos, 91 396 caracteres, ~22.5K tokens
Tier enrutado (determinista):	deepseek (deepseek/deepseek-v4-pro)
Chunks destilados:	2
Validación neuro-simbólica:	0 errores (74 bloques; 0 peligro, el resto ok/observacion)
Estructuras de referencia:	6/6 presentes (api, patrones, ejemplos, configuracion, prerequisites, glosario)
Reporte LaTeX:	docs/SKILL/pallets_itsdangerous/ (.tex + .pdf)
Coste de la fase:	~\$0.01 USD
Instalación:	Opcional (con confirmación del usuario; ya instalado en global)

Fortaleza de diseño

El **gate de instalación**: el orquestador solo instala en global si la validación del paso 5 no arroja bloques peligrosos. Con `-dry-run` genera y valida sin tocar el entorno de producción, y `-reflexion` permite corregir iterativamente antes de instalar. RAS: reproducibilidad, auditabilidad y seguridad en cada ejecución.

7 >_ Guía de Invocación Rápida

>_ Ejemplos de uso del orquestador

```
1 # 1. Flujo completo (entrevista interactiva si faltan datos)
2 python3 flujo_repo_a_skill.py --repo https://github.com/pallets/
  itsdangerous \
3   --tema "Firma y autenticacion de datos" --nombre
  pallets_itsdangerous
4
5 # 2. Modo dry-run: genera y valida sin instalar en global
6 python3 flujo_repo_a_skill.py --repo /ruta/al/repo --nombre
  mi_skill --dry-run
7
8 # 3. Acotar el alcance del repo con globs (controla coste)
9 python3 flujo_repo_a_skill.py --repo https://github.com/usuario/
  repo \
10  --incluir "src/**" "*.py" --excluir "**/tests/*" --nombre
  mi_skill
11
12 # 4. Todo explicito, sin instalacion (para revisar antes de
  publicar)
13 python3 flujo_repo_a_skill.py --repo /ruta --nombre mi_skill --tema
  "API de ejemplo" \
14  --idioma es --dry-run --validar-estricto --reflexion 1
```

Recuerda

Tras instalar un skill, **reinicia opencode**: la configuración no se recarga en caliente y el skill no será detectable hasta el reinicio. El orquestador ya recuerda este aviso en su último paso (alerta success).